Chapter 6. Image Recognition

Topic: Object detection survey

Ph.D Nguyen Thi Khanh Tiên

4-2023

Object detection

Object detection is a computer vision technique to localize and classify objects in an image or video.

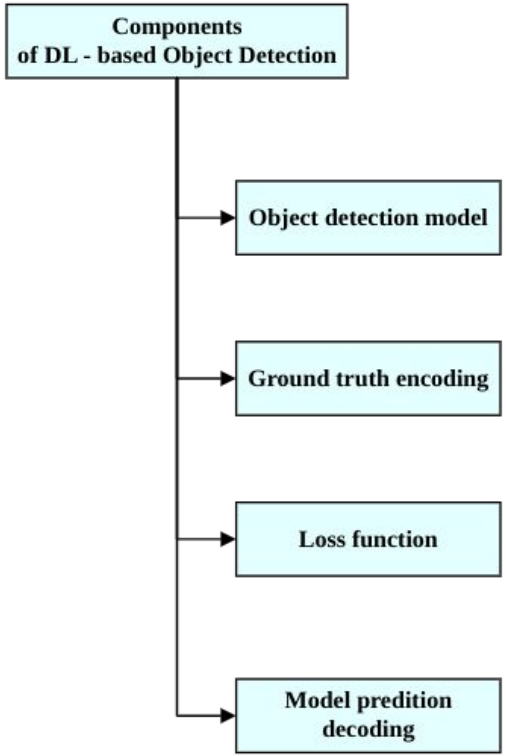
Object detection has the following four components :

– Object detection model

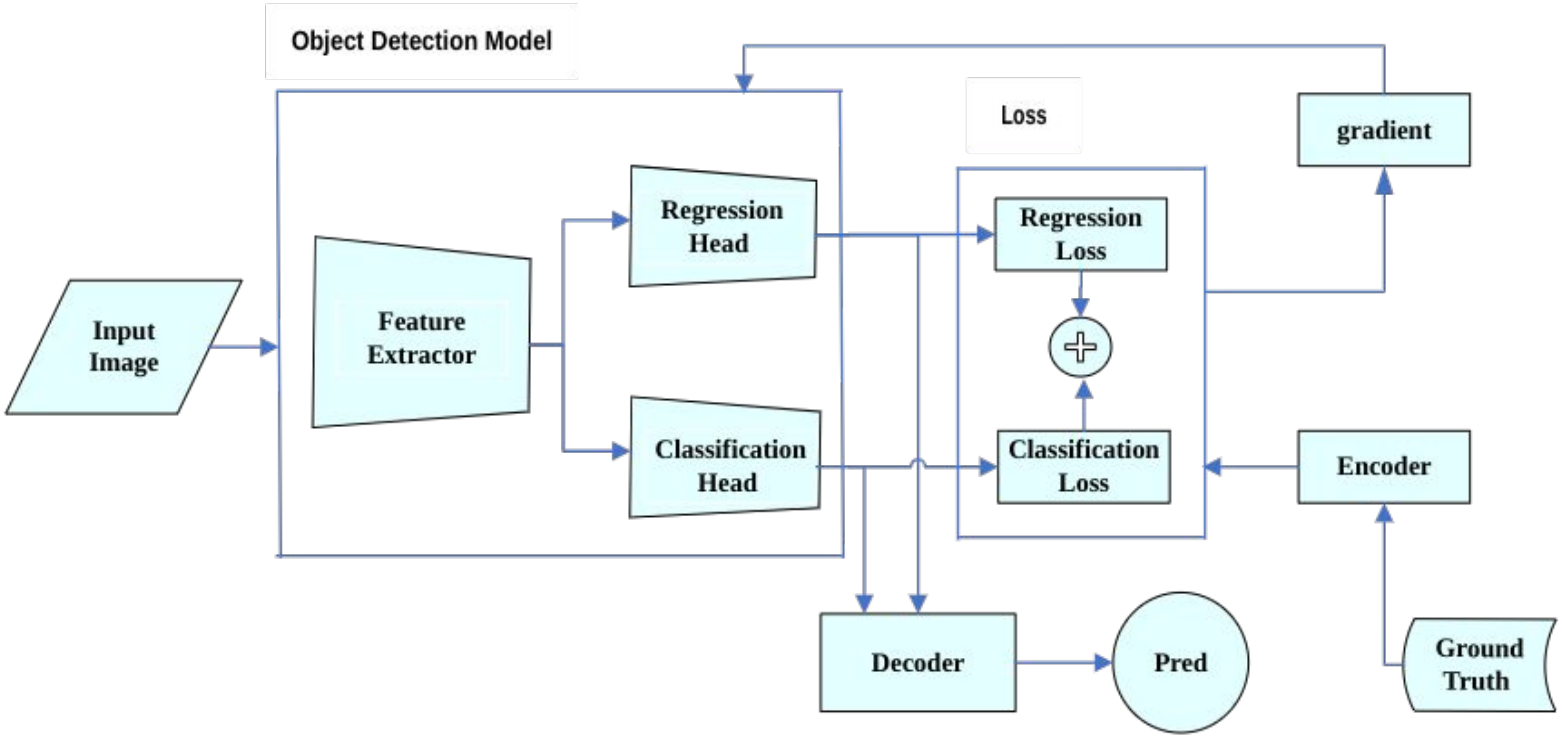
– Ground truth encoding

– Loss function

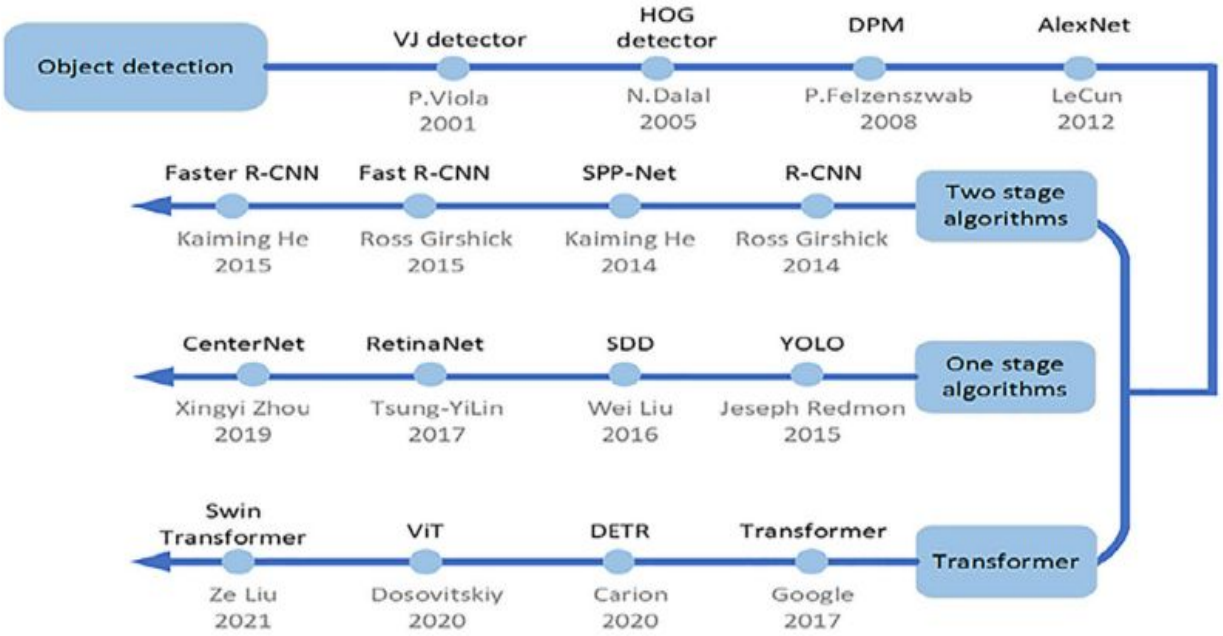
– Model prediction decoding



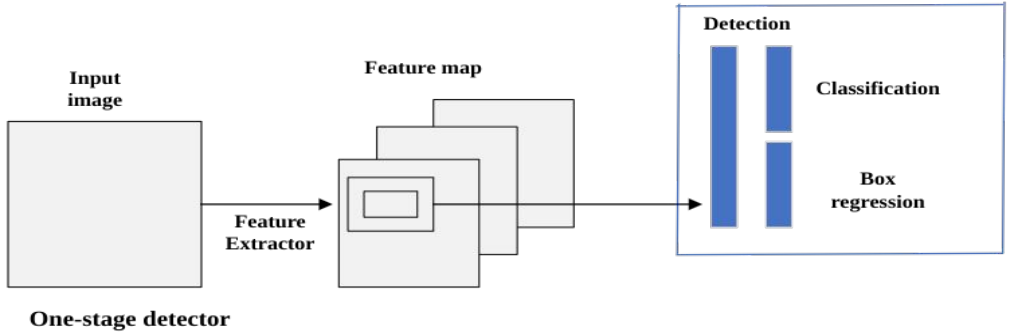
Object Detection Model Training/Inference Pipeline

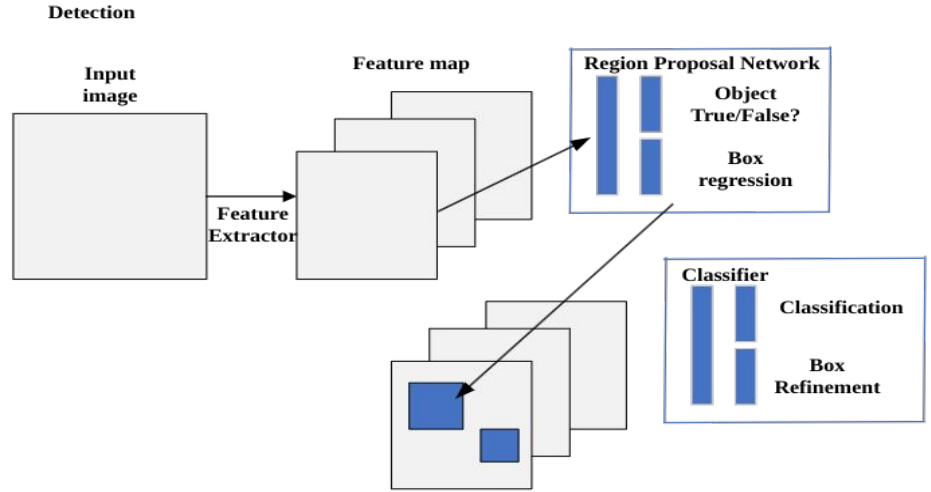


OBJECT DETECTORS.



One-stage and two-stage object detectors





Backbone Architectures.

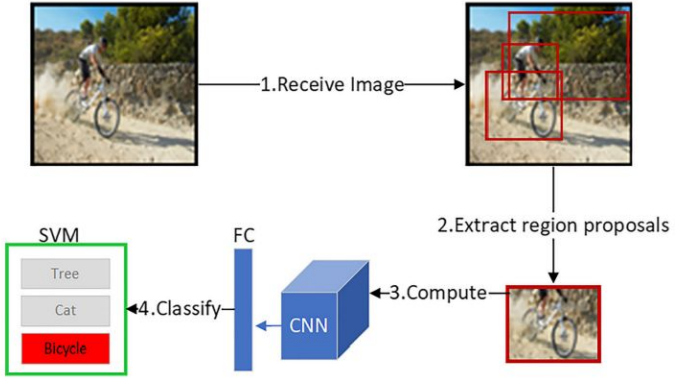
1. AlexNet
2. VGG
3. GoogleNet/Inception
4. ResNet
5. CSPNet
6. MobileNet
7. EfficientNet

- The detection steps of the two-stage algorithms are mainly as follows: first extract features, second extract region proposal, and third is classification, positioning.
- Two stage algorithms results' accuracy relatively high, but the detection speed is slow.
- The one-stage algorithms directly generate the positioning coordinates and classification probability of objects in the image, without the need to generate region proposal in advance.
- Because there is one less computationally intensive step, their detection speed is very fast, and the detection result can be obtained directly.

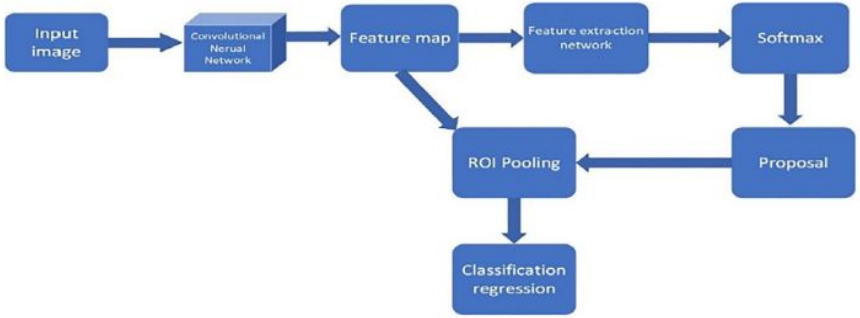
R-CNN Model Family

The R-CNN Model family includes the following:

1. ***R-CNN*** utilizes a selective search method to locate RoIs in the input images and uses a DCNN, based region wise classifier to classify the RoIs independently.
2. ***SPPNet*** and ***Fast R-CNN*** - the improved versions of R-CNN that deals with the extraction of the RoIs from the feature maps. This was found to be much faster than the conventional R-CNN architecture.
3. ***Faster R-CNN*** - the improved version of Fast R-CNN that was trained end to end by introducing RPN (region proposal network). An RPN is a network utilized in generating RoIs by regressing the anchor boxes. Hence, the anchor boxes are then used in the object detection task.
4. ***Mask R-CNN*** adds a mask prediction branch on the Faster R-CNN, which can detect objects and predict their masks at the same time.
5. ***R-FCN*** replaces the fully connected layers with the position-sensitive score maps for better detecting objects.
6. ***Cascade R-CNN*** addresses the problem of overfitting at training and quality mismatch at inference by training a sequence of detectors with increasing IoU thresholds.



The four steps of R-CNN



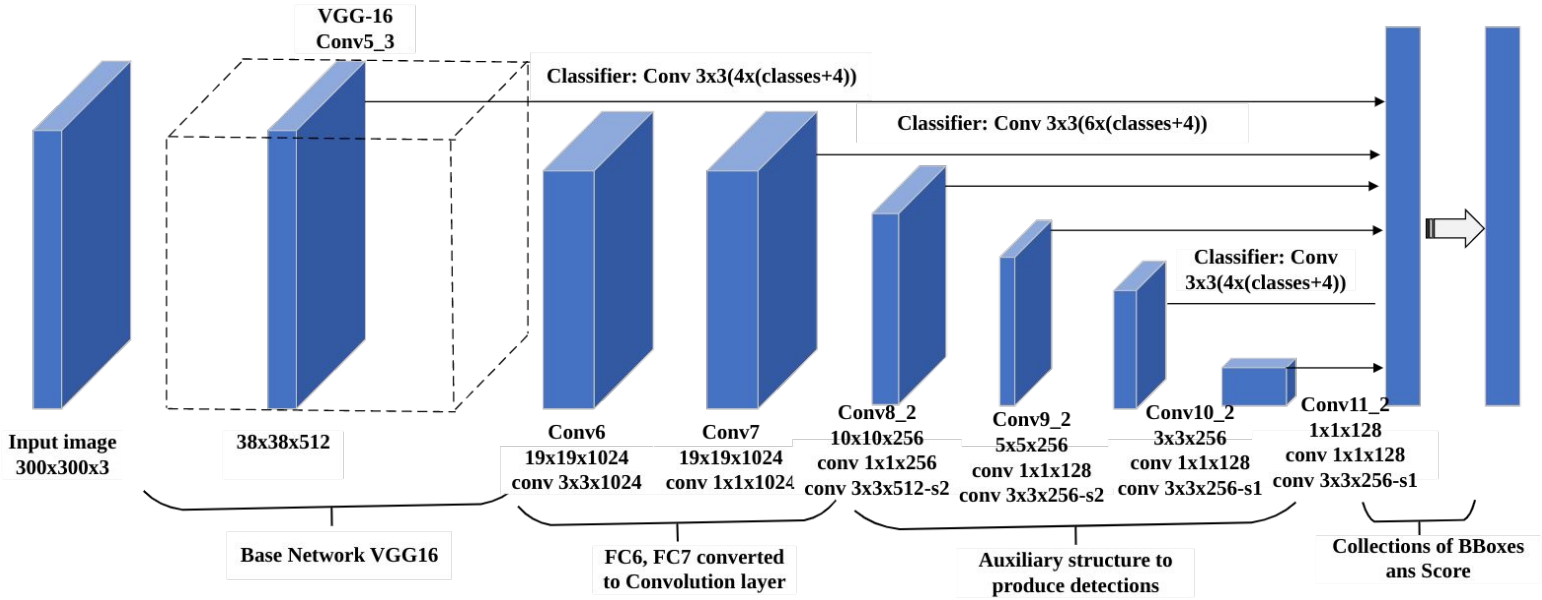
Detection process of Faster R-CNN

SSD model architecture

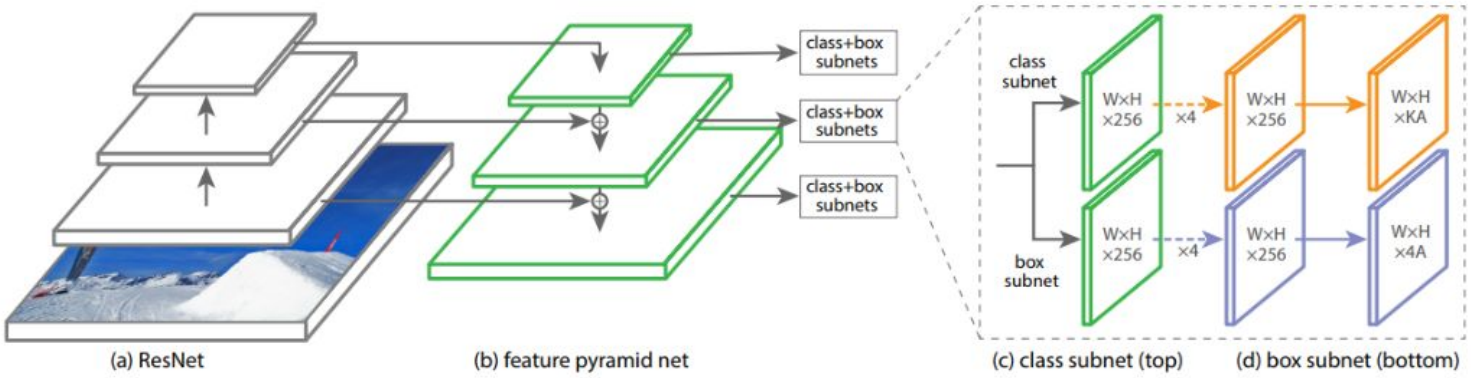
SSD places anchor boxes densely over an input image and uses features from different convolutional layers to regress and classify the anchor boxes.

The SSD has two components: the backbone model and the SSD head.

In figure, the SSD model is built with VGG on a Resnet pre-trained model that is converted to a fully CNN.



Retina Net model architecture



RetinaNet is one of the best one-stage object detection models that has proven to work well with dense and small scale objects. This detection model has been formed by making two improvements over existing single stage object detection models - ***Feature Pyramid Networks (FPN)*** and ***Focal Loss.***

There are four major components of a RetinaNet model architecture:

a) ***Bottom-up Pathway*** - The backbone network (e.g. ResNet) which calculates the feature maps at different scales, irrespective of the input image size or the backbone.

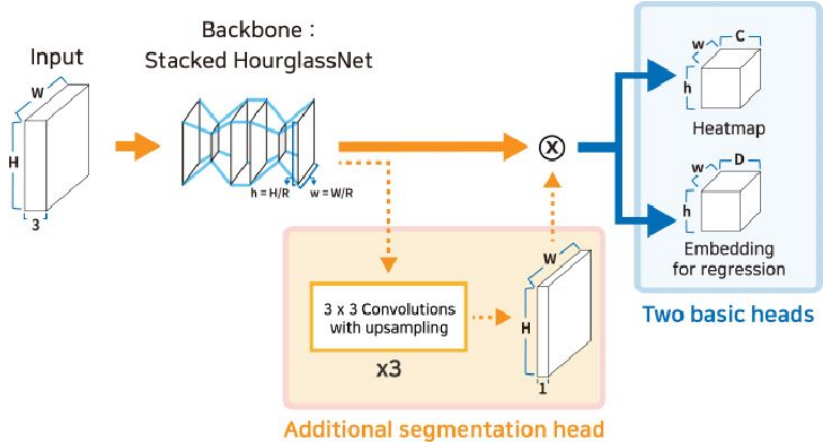
b) ***Top-down pathway*** and Lateral connections - The top down pathway up samples the spatially coarser feature maps from higher pyramid levels, and the lateral connections merge the top-down layers and the bottom-up layers with the same spatial size.

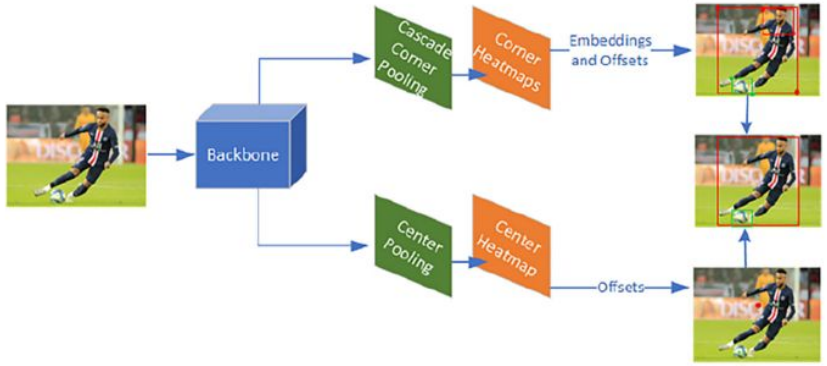
c) ***Classification subnetwork*** - It predicts the probability of an object being present at each spatial location for each anchor box and object class.

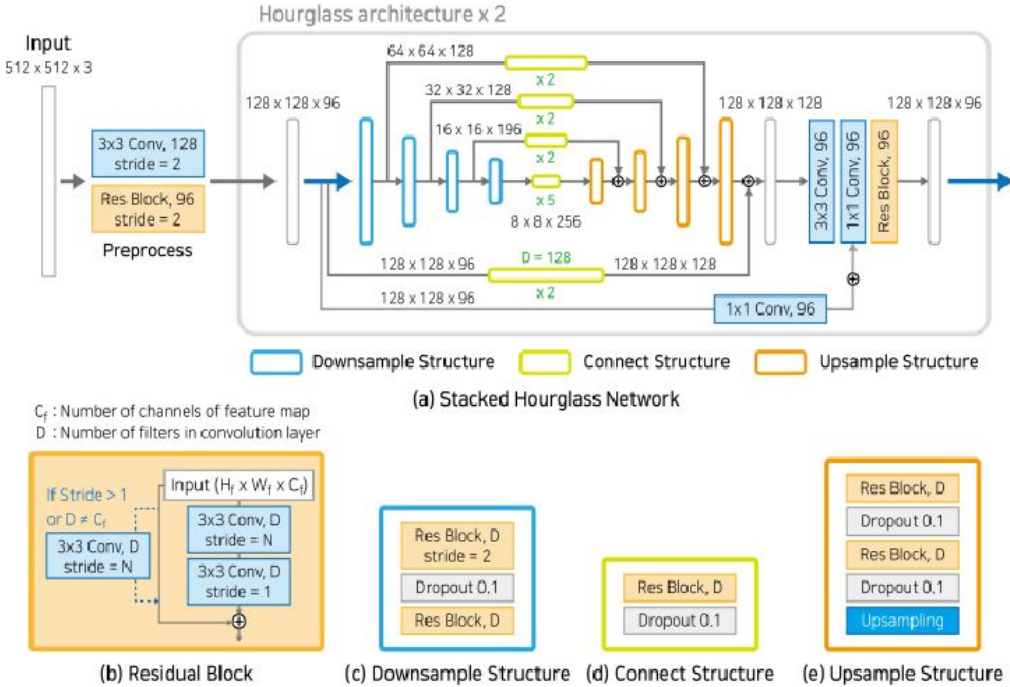
d) ***Regression subnetwork*** - It's regresses the offset for the bounding boxes from the anchor boxes for each ground-truth object.

CenterNet

CenterNet explores the visual patterns within each bounding box. CenterNet evaluates objects as single points by predicting the x and y coordinate of the object's center and it's area of coverage (width and height). It is a unique technique that has proven to out-perform variants like the SSD and R-CNN family.



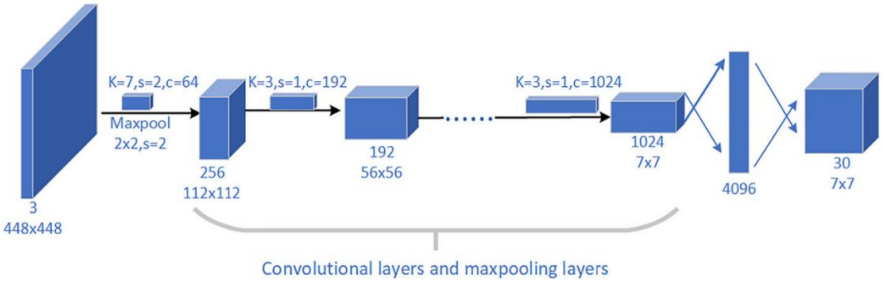




YOLO Model Family

The YOLO family model includes the following:

- ***YOLOv1*** uses fewer anchor boxes (divide the input image into an S × S grid) to do regression and classification. This was built using darknet neural networks.
- ***YOLOv2*** improves the performance by using more anchor boxes and a new bounding box regression method. They designed a brand new backbone network called ***Darknet-19*** which included 19 convolutional layers and 5 max pooling layers.
- ***YOLOv3*** is an enhanced version of the v2 variant with a deeper feature detector network and minor representational changes. The backbone of YOLOv3 is a ***residual model*** which called ***Darknet-53.*** YOLOv3 has relatively speedy inference times with it taking roughly 30ms per inference.
- ***YOLOv4*** (YOLOv3 upgrade) had been improved in many ways, like data processing, network training, activation function, loss function, etc. YOLO V4 and its successors are technically the product of a



Structure of YOLOv1 almost has no difference from other ordinary classification network

- ***YOLOv5*** is an improved version of YOLOv4 with a mosaic augmentation technique for increasing the general performance of YOLOv4.
- ***YOLOv8*** is the newest state-of-the-art YOLO model that can be used for object detection, image classification, and instance segmentation tasks. YOLOv8 was developed by Ultralytics, who also created the influential and industry-defining YOLOv5 model. YOLOv8 includes numerous architectural and developer experience changes and improvements over YOLOv5.

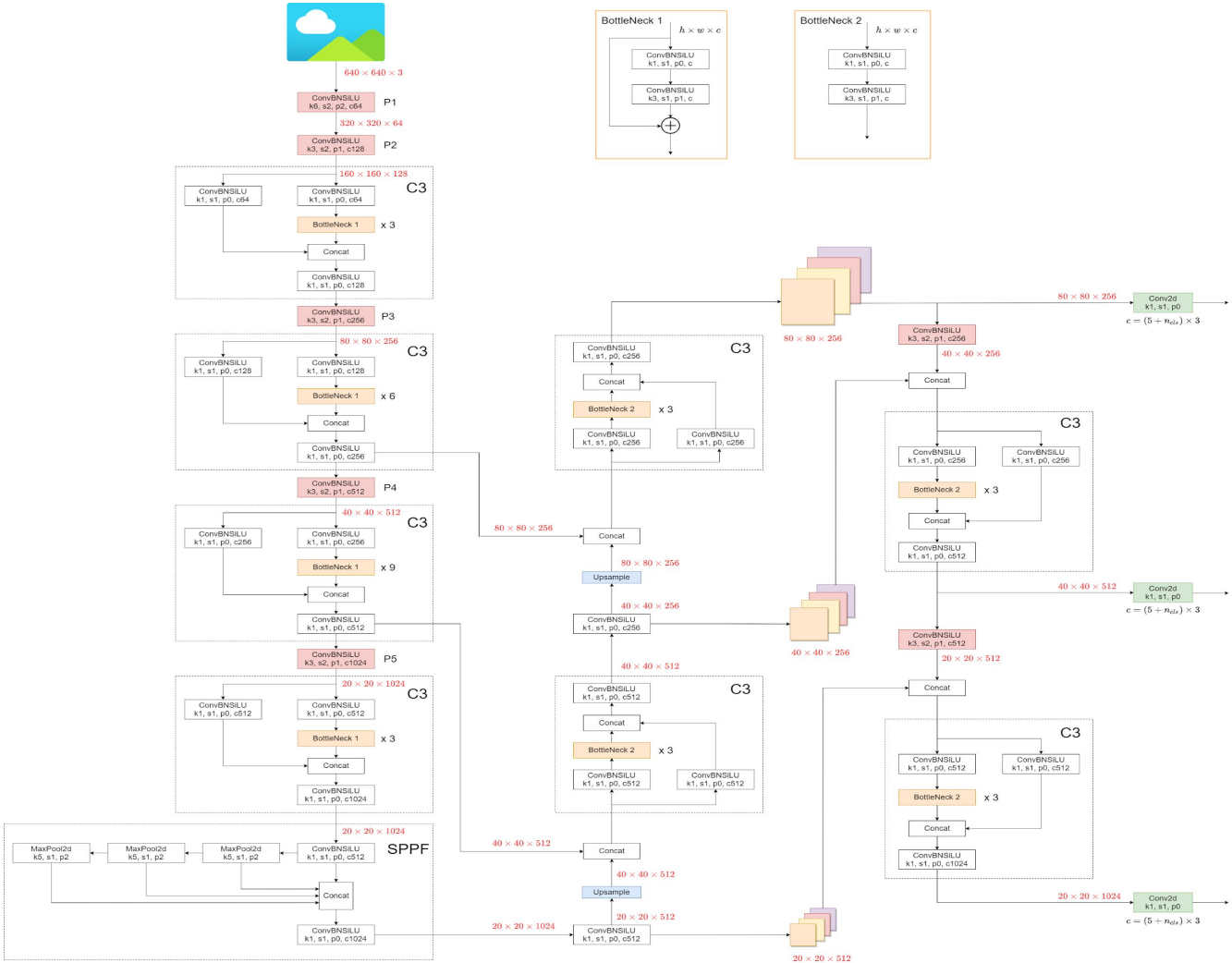
Model architecture summary of YOLOv3, YOLOv4, and YOLOv5.

The YOLO framework has three main components:

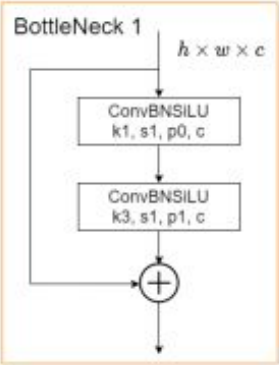
- ***Backbone***. The Backbone mainly extracts essential features of an image and feeds them to the Head through Neck.
- ***Neck.***The Neck collects feature maps extracted by the Backbone and creates feature pyramids.
- ***Head***. The head consists of output layers

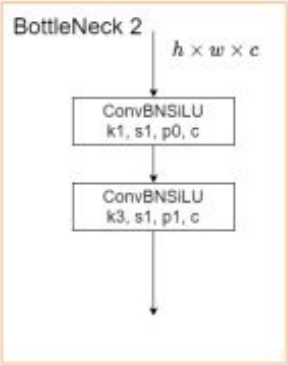
Layers	YOLOv3	YOLOv4	YOLOv5
Neural Network Type	FCNN	FCNN	FCNN
Backbone	**Darknet53**	**CSPDarknet53** (CSPNet in Darknet)	**CSPDarkent53** Focus structure
Neck	**FPN** (Feature Pyramid Network)	**SPP** (Spatial Pyramid Pooling) and **PANet** (Path Aggregation Network)	**PANet**
Head	**B x (5 + C)** output layer **B**: No. of bounding boxes **C**: Class score	Same as Yolo v3	Same as Yolo v3
Loss Function	Binary Cross Entropy	Binary Cross Entropy	Binary Cross Entropy and Logit Loss Function

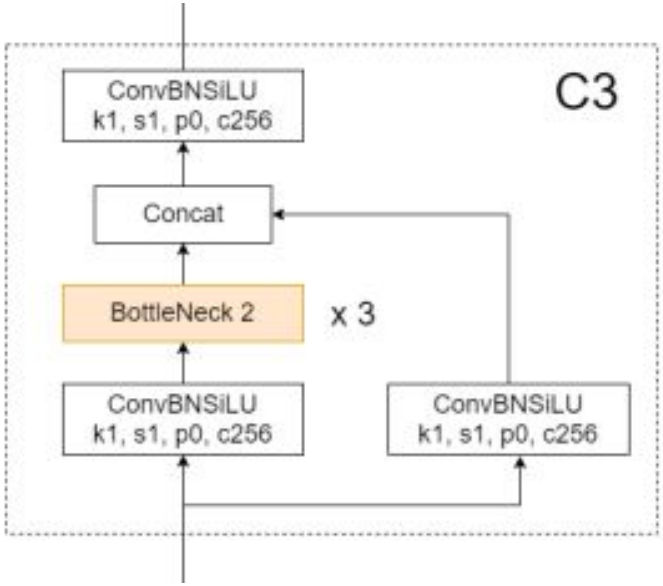
YOLOv5

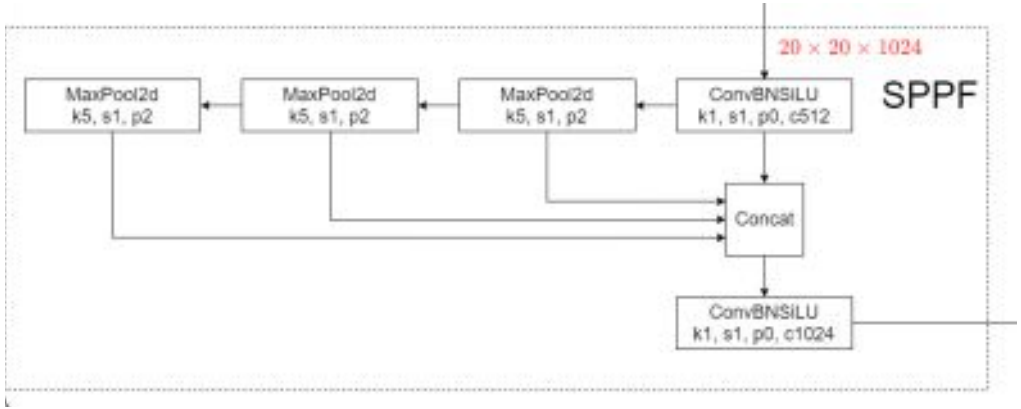

BottleNeck 1
$h \times w \times c$
ConvBNSiLU k1, s1, p0, c
ConvBNSiLU k3, s1, p1, c
BottleNeck 2
$h \times w \times c$
ConvBNSiLU k1, s1, p0, c
ConvBNSiLU k3, s1, p1, c
$640 \times 640 \times 3$
ConvBNSiLU k6, s2, p2, c64
P1
$320 \times 320 \times 64$
ConvBNSiLU k3, s2, p1, c128
P2
$160 \times 160 \times 128$
C3
ConvBNSiLU k1, s1, p0, c64
ConvBNSiLU k1, s1, p0, c64
BottleNeck 1
x 3
Concat
ConvBNSiLU k1, s1, p0, c128
ConvBNSiLU k3, s2, p1, c256
P3
$80 \times 80 \times 256$
C3
ConvBNSiLU k1, s1, p0, c128
ConvBNSiLU k1, s1, p0, c128
BottleNeck 1
x 6
Concat
ConvBNSiLU k1, s1, p0, c256
ConvBNSiLU k3, s2, p1, c512
P4
$40 \times 40 \times 512$
C3
ConvBNSiLU k1, s1, p0, c256
ConvBNSiLU k1, s1, p0, c256
BottleNeck 1
x 9
Concat
ConvBNSiLU k1, s1, p0, c512
ConvBNSiLU k3, s2, p1, c1024
P5
$20 \times 20 \times 1024$
C3
ConvBNSiLU k1, s1, p0, c512
ConvBNSiLU k1, s1, p0, c512
BottleNeck 1
x 3
Concat
ConvBNSiLU k1, s1, p0, c1024
$20 \times 20 \times 1024$
SPPF
MaxPool2d k5, s1, p2
MaxPool2d k5, s1, p2
MaxPool2d k5, s1, p2
ConvBNSiLU k1, s1, p0, c512
Concat
ConvBNSiLU k1, s1, p0, c1024
$80 \times 80 \times 256$
Concat
Upsample
$40 \times 40 \times 256$
ConvBNSiLU k1, s1, p0, c256
C3
ConvBNSiLU k1, s1, p0, c256
Concat
BottleNeck 2
x 3
ConvBNSiLU k1, s1, p0, c256
ConvBNSiLU k1, s1, p0, c256
$40 \times 40 \times 512$
Concat
Upsample
$20 \times 20 \times 512$
ConvBNSiLU k1, s1, p0, c512
C3
ConvBNSiLU k1, s1, p0, c512
Concat
BottleNeck 2
x 3
ConvBNSiLU k1, s1, p0, c512
ConvBNSiLU k1, s1, p0, c512
$80 \times 80 \times 256$
$80 \times 80 \times 256$
Conv2d k1, s1, p0
$c = (5 + n_{cls}) \times 3$
ConvBNSiLU k3, s2, p1, c256
$40 \times 40 \times 256$
Concat
C3
ConvBNSiLU k1, s1, p0, c256
ConvBNSiLU k1, s1, p0, c256
BottleNeck 2
x 3
Concat
ConvBNSiLU k1, s1, p0, c512
$40 \times 40 \times 512$
Conv2d k1, s1, p0
$c = (5 + n_{cls}) \times 3$
$40 \times 40 \times 256$
ConvBNSiLU k3, s2, p1, c512
$20 \times 20 \times 512$
Concat
C3
ConvBNSiLU k1, s1, p0, c512
ConvBNSiLU k1, s1, p0, c512
BottleNeck 2
x 3
Concat
ConvBNSiLU k1, s1, p0, c1024
$20 \times 20 \times 1024$
Conv2d k1, s1, p0
$c = (5 + n_{cls}) \times 3$
$20 \times 20 \times 512$

YOLOv5

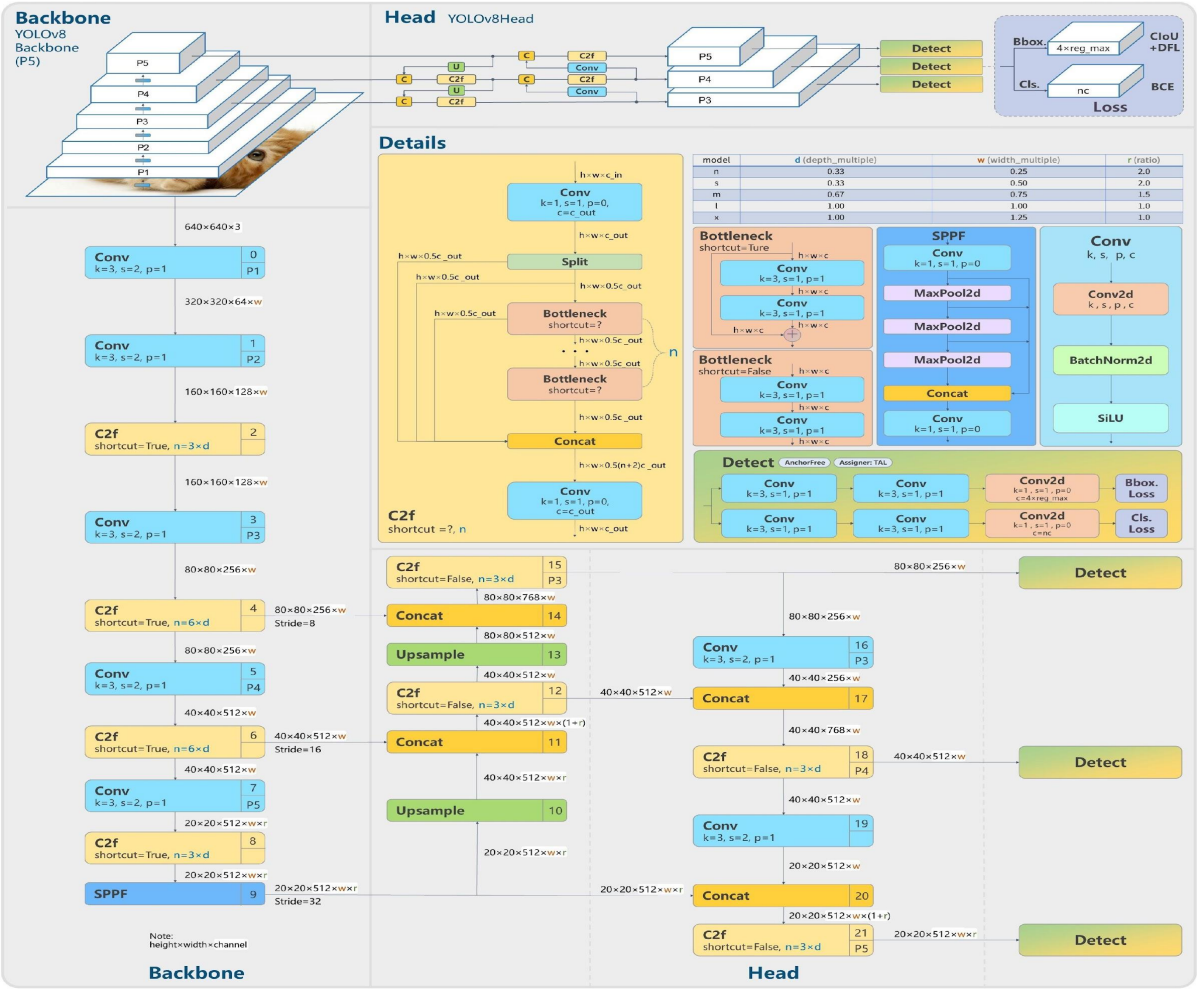

BottleNeck 1
$h \times w \times c$
ConvBNSiLU
k1, s1, p0, c
ConvBNSiLU
k3, s1, p1, c


BottleNeck 2
$h \times w \times c$
ConvBNSiLU
k1, s1, p0, c
ConvBNSiLU
k3, s1, p1, c


C3
ConvBNSiLU
k1, s1, p0, c256
Concat
BottleNeck 2
x 3
ConvBNSiLU
k1, s1, p0, c256
ConvBNSiLU
k1, s1, p0, c256

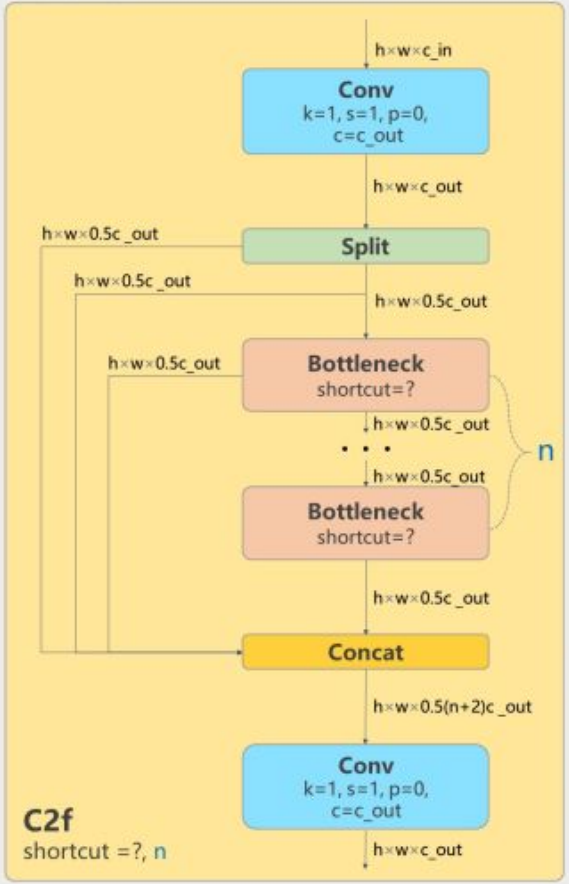

$20 \times 20 \times 1024$
SPPF
MaxPool2d
k5, s1, p2
MaxPool2d
k5, s1, p2
MaxPool2d
k5, s1, p2
ConvBNSiLU
k1, s1, p0, c512
Concat
ConvBNSiLU
k1, s1, p0, c1024

YOLOv8

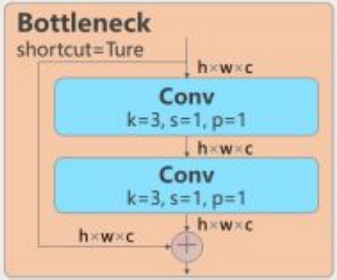

Backbone
YOLOv8 Backbone (P5)
P5
P4
P3
P2
P1
640×640×3
Conv k=3, s=2, p=1 | 0 | P1
320×320×64×w
Conv k=3, s=2, p=1 | 1 | P2
160×160×128×w
C2f shortcut=True, n=3×d | 2
160×160×128×w
Conv k=3, s=2, p=1 | 3 | P3
80×80×256×w
C2f shortcut=True, n=6×d | 4
80×80×256×w Stride=8
80×80×256×w
Conv k=3, s=2, p=1 | 5 | P4
40×40×512×w
C2f shortcut=True, n=6×d | 6
40×40×512×w Stride=16
40×40×512×w
Conv k=3, s=2, p=1 | 7 | P5
20×20×512×w×r
C2f shortcut=True, n=3×d | 8
20×20×512×w×r
SPPF | 9
20×20×512×w×r Stride=32
Note: height×width×channel
Backbone
Head YOLOv8Head
U
C
C2f
Conv
P5
P4
P3
Detect
Detect
Detect
Bbox.
4×reg_max
CIoU +DFL
Cls.
nc
BCE
Loss
Details
h×w×c_in
Conv k=1, s=1, p=0, c=c_out
h×w×c_out
Split
h×w×0.5c_out
Bottleneck shortcut=?
n
Concat
h×w×0.5(n+2)c_out
C2f shortcut =?, n
model | d (depth_multiple) | w (width_multiple) | r (ratio)
n | 0.33 | 0.25 | 2.0
s | 0.33 | 0.50 | 2.0
m | 0.67 | 0.75 | 1.5
l | 1.00 | 1.00 | 1.0
x | 1.00 | 1.25 | 1.0
Bottleneck shortcut=Ture
Conv k=3, s=1, p=1
Bottleneck shortcut=False
SPPF
Conv k=1, s=1, p=0
MaxPool2d
Concat
Conv k, s, p, c
Conv2d k, s, p, c
BatchNorm2d
SiLU
Detect AnchorFree Assigner: TAL
Conv k=3, s=1, p=1
Conv2d k=1, s=1, p=0 c=4×reg_max
Conv2d k=1, s=1, p=0 c=nc
Bbox. Loss
Cls. Loss
C2f shortcut=False, n=3×d | 15 | P3
80×80×768×w
Concat | 14
80×80×512×w
Upsample | 13
40×40×512×w
C2f shortcut=False, n=3×d | 12
40×40×512×w×(1+r)
Concat | 11
40×40×512×w×r
Upsample | 10
20×20×512×w×r
80×80×256×w
Detect
Conv k=3, s=2, p=1 | 16 | P3
40×40×256×w
Concat | 17
40×40×768×w
C2f shortcut=False, n=3×d | 18 | P4
40×40×512×w
Detect
Conv k=3, s=2, p=1 | 19
20×20×512×w
Concat | 20
20×20×512×w×(1+r)
C2f shortcut=False, n=3×d | 21 | P5
20×20×512×w×r
Detect
Head

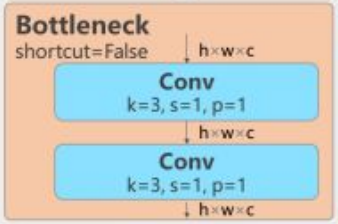
YOLOv8

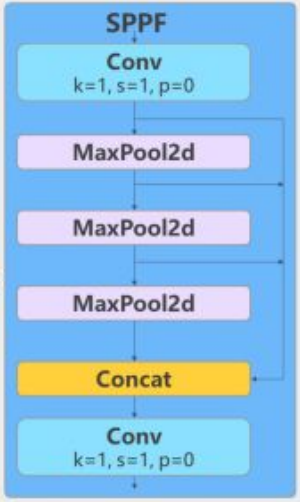
Details

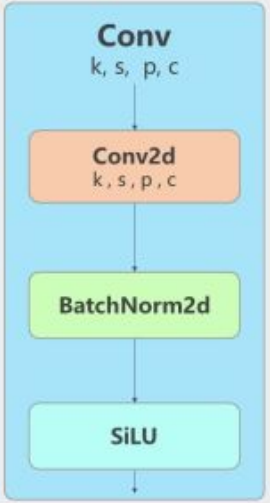


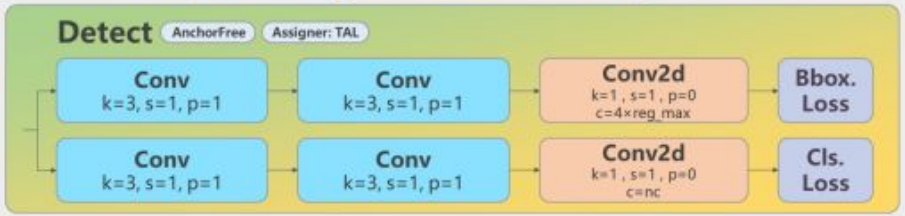
model	d (depth_multiple)	w (width_multiple)	r (ratio)
n	0.33	0.25	2.0
s	0.33	0.50	2.0
m	0.67	0.75	1.5
l	1.00	1.00	1.0
x	1.00	1.25	1.0



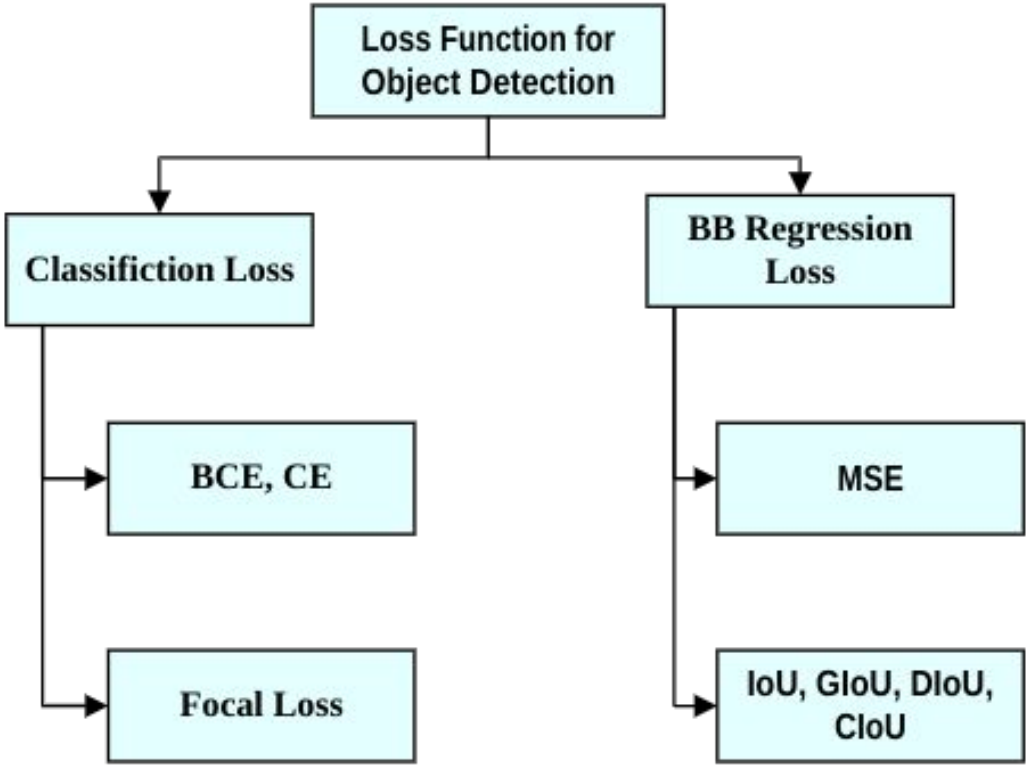








Loss Function for Object Detection



Classification Loss for Object Detection

- ***Cross-Entropy (CE) loss*** is traditionally used in classification tasks.

- Cases where Cross-Entropy loss performs badly:

  -Class imbalance inherits bias in the process.

  -Fails to distinguish between hard and easy examples.

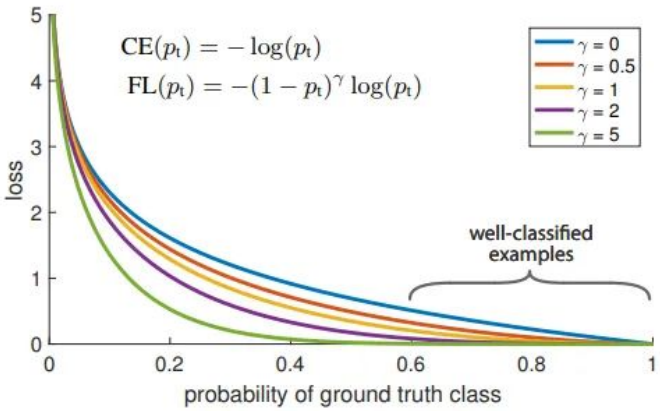
- ***Focal Loss (FL)*** applies a modulating term to the cross-entropy loss in order to focus learning on hard misclassified examples

$$FL(p_t) = -\alpha(1-p_t)^{\gamma} \cdot log(p_t)$$

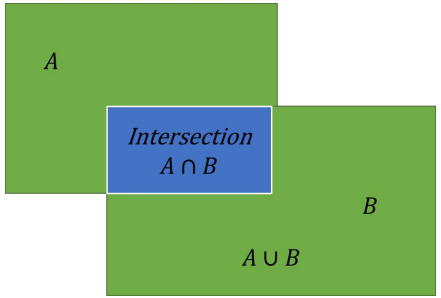
γ offsets the class imbalance of the number of examples. This alpha is set inversely proportional to the number of examples for a p_t arti p_t lar class or is learned through cross-validation. α ∈[0,1]

$(1-p_t)^{\gamma}$ to focus on the hard examples. The hard p_t and easy examples are defined by the probability of the correct classification, . If is low, then we are dealing with a hard sample. In this case, the modulating factor, remains near 1 and is unaffected. If is high, then the

$$p_t = \begin{cases} p & \text{if } y = 1 \\ 1-p & \text{otherwise,} \end{cases}$$



Bounding Box Regression Loss Function for Object Detection

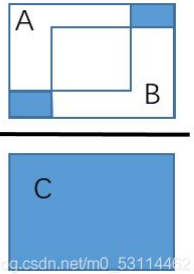


- Intersection over Union $IoU = \frac{A \cap B}{A \cup B}$

- GIoU (General IoU)

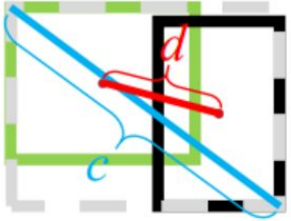
$$L_{GIoU} = 1 - IoU + \frac{|C \backslash (A \cup B)|}{|C|}$$

GIoU=IoU-$\frac{|C/A\cup B|}{|C|}$ = IoU -

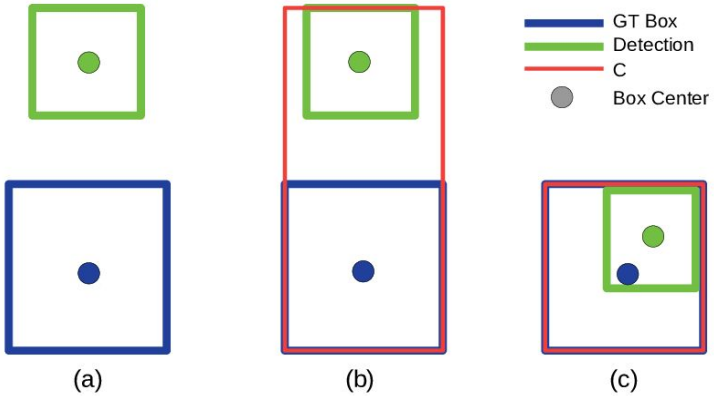


https://blog.csdn.net/m0_53114482

- DIoU $L_{DIoU} = 1 - IoU + \frac{p^2(a,b)}{c^2}$



- CIoU $L_{CIoU} = 1 - IoU + \frac{p^2(a,b)}{c^2} + \alpha v$



where:

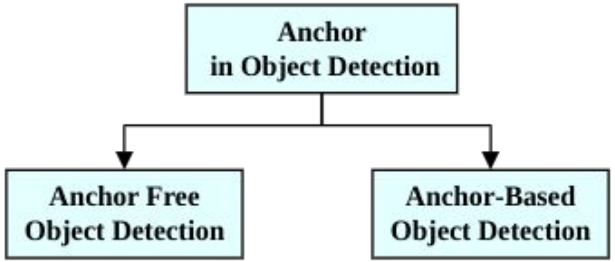
- IoU(A,B) < 0.5 => $\alpha = 0$
- IoU(A,B) >= 0.5 => $\alpha = \frac{v}{(1 - IoU) + v}$,

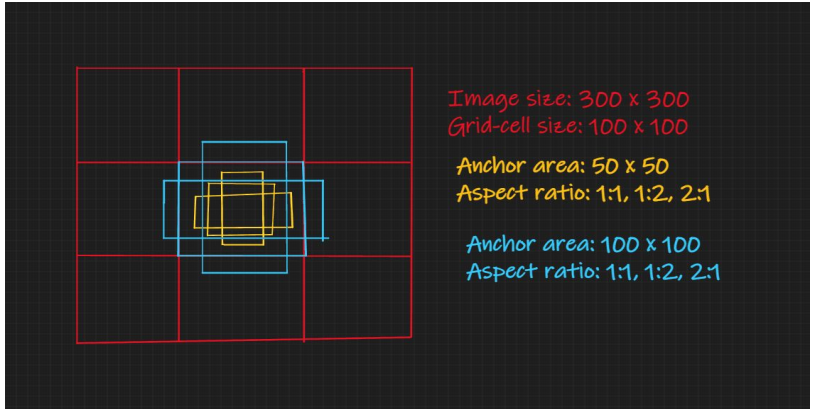
c– diagonal length of the smallest enclosing box covering two box.

d=p(a,b) - distance of central points of predicted and ground-truth boxes

Anchor in Object Detection

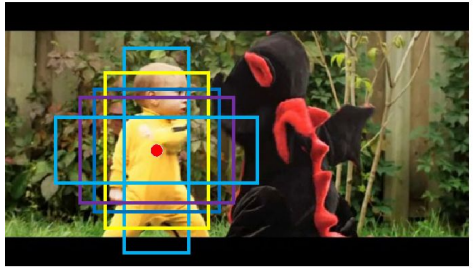
In object detection, anchors are pre-defined bounding boxes as proposals of ground truth. Normally, the image will be divided into grid cells. In each of these grid cells the predefined bounding boxes defined from the center of the grid cell are called anchors. A grid cell can contain multiple anchors.





In ***anchor-based object detection***, the model predicts the class labels of anchor boxes and how the anchor should be adjusted to match it with ground truth objects.

Anchor free object detection directly predicts the bounding box, but it predicts it with respect to some fixed reference in the image.



Anchor-based

Anchor-free

Following are a few advantages of anchor free methods over anchor-based:

– Finding suitable anchor boxes (in shape and size) is crucial in training an excellent anchor-based object detection model. Finding suitable anchors is a complex problem and may need hyper-parameter tuning.

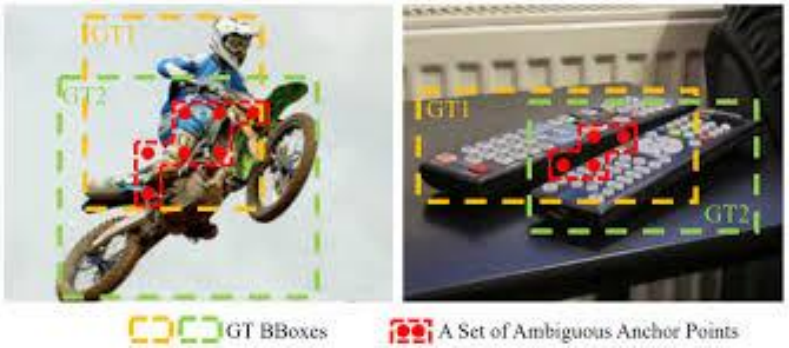
– Using more anchors results in better accuracy in anchor-based object detection but using more anchors comes at a cost. The model needs more complex architecture, which leads to slower inference speed.

Label Assigment

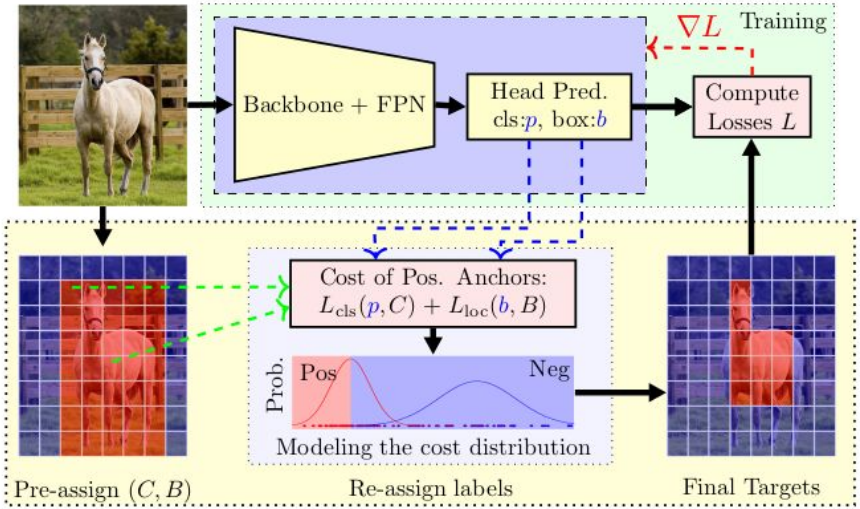
- Label assignment is to divide the samples into positives and negatives, which is essential to the success of object detection models.

Illustration of the defects of the conventional label assignment in current one-stage detectors. They suffer from two major limits:
(1) Misalignment between assignment strategy and NMS procedure.
(2) Rigid sample assignment during training process.

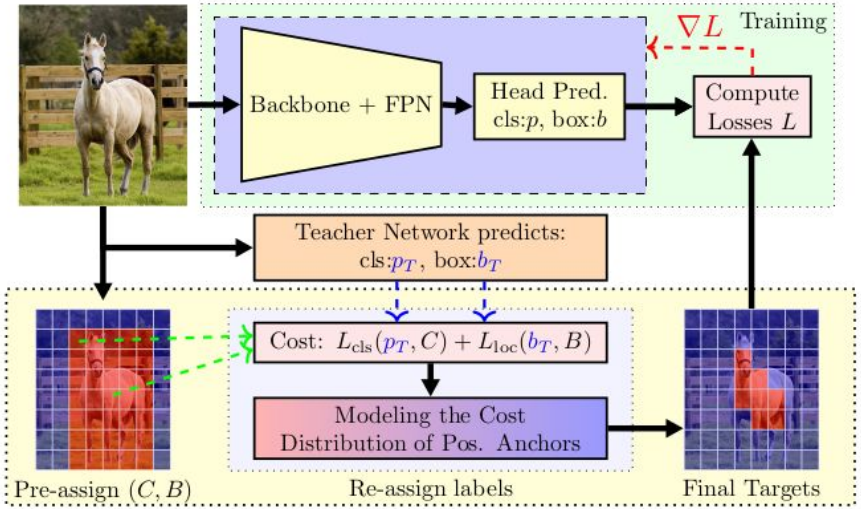
In the training process, the label assignment strategy plays an important role and it is responsible for separating positive and negative samples.



Label Assigment



Probabilistic Anchor Assignment (PAA)



Label Assignment Distillation (LAD)

Metrics used for Evaluating Object Detection Models

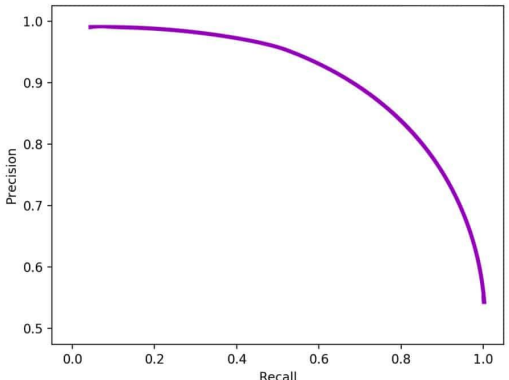
Intersection over Union (IoU) is a metric that quantifies the degree of overlap between two regions to evaluate the accuracy of a prediction.

Precision. Accuracy measures the proportion of positives that are predicted to be correct.

$$P = TP / (TP + FP) = TP / Total\ Prediction$$

Recall is a parameter that represents the ratio of correct predictions to the total number of ground truths.

$$R = TP / (TP + FN) = TP / Total\ Ground\ Truths$$



Precision-Recall Curve (PR-Curse) is a graph with Precision rate on the y-axis and Recall rate on the x-axis . It shows the precision as a recall function for all the different threshold values.

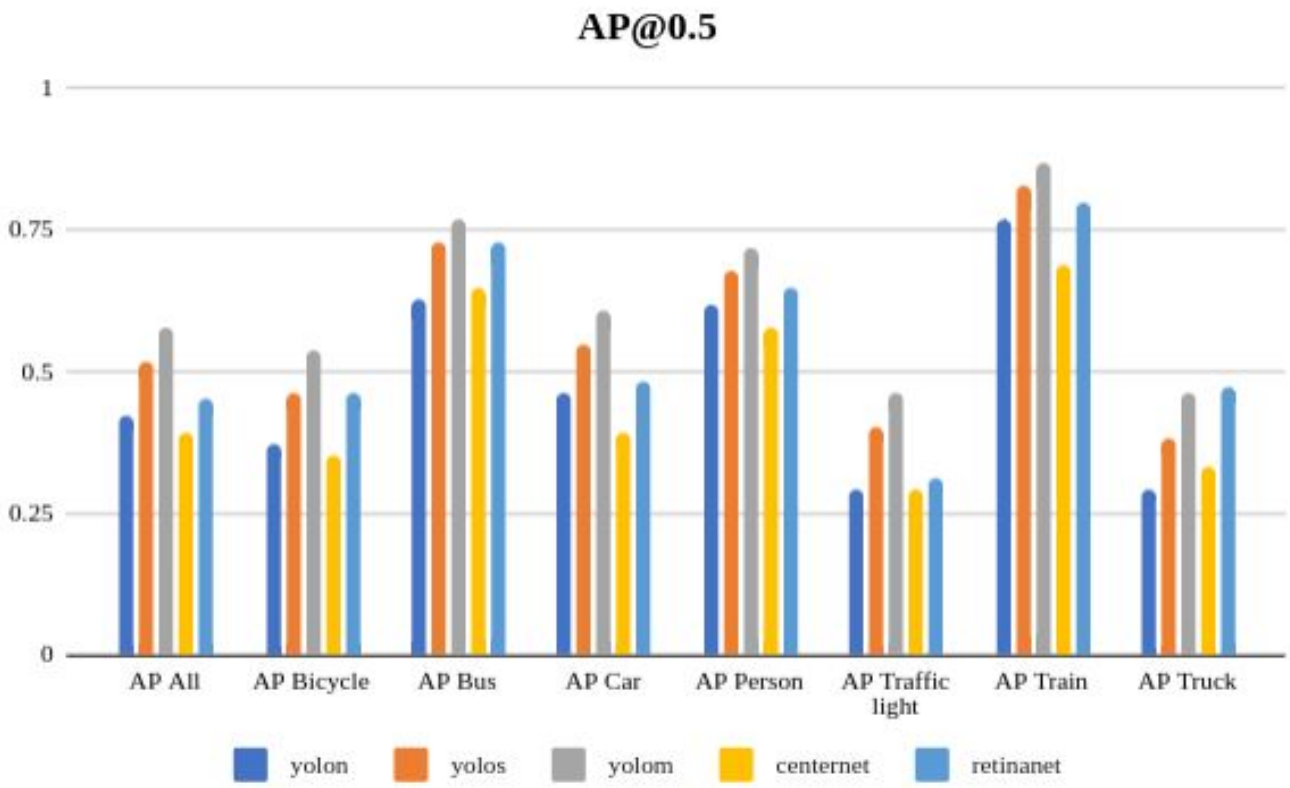
Average Precision (AP) is not the mean of precision, it's the area under the PR Curse. The area under the curve used to summarize the performance of a model becomes a general measure of prediction accuracy.

Mean Average Precision (mAP) is the average of AP over all classes detected in multilayer object detection

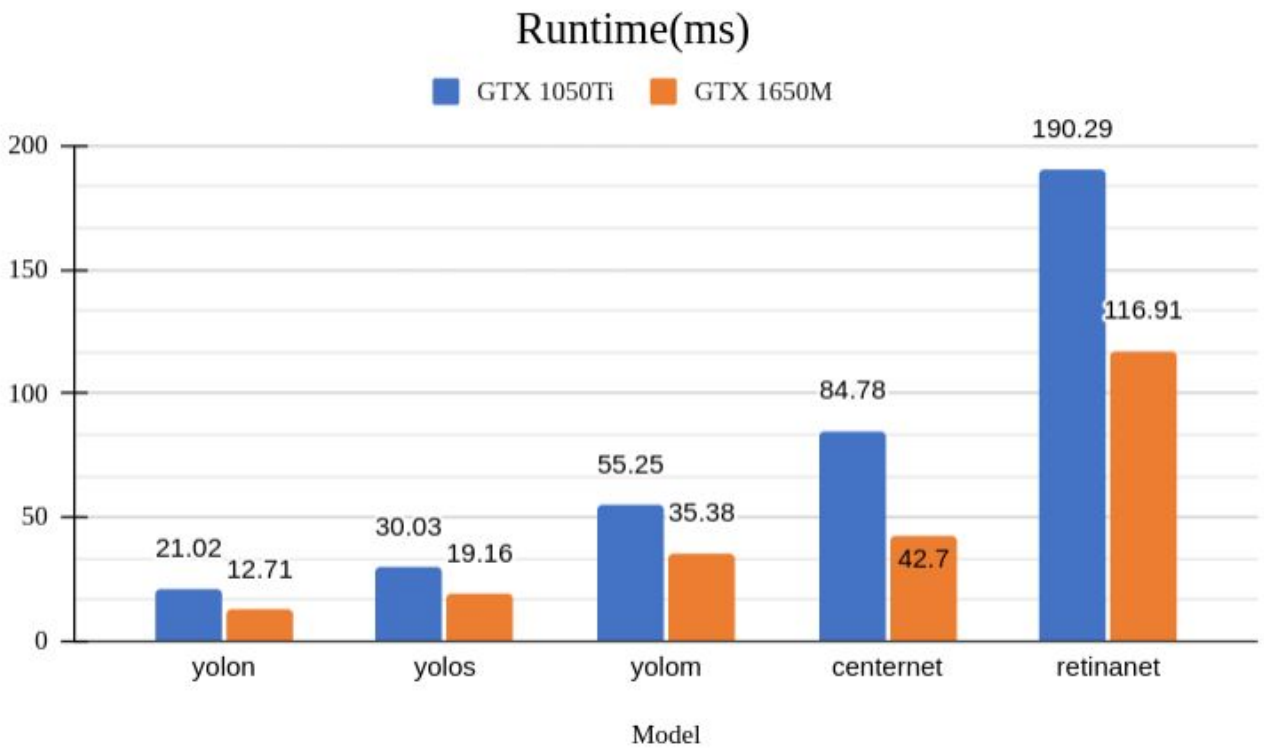
$$mAP = 1/n * sum(AP)$$

The precision and recall values change when the IoU threshold changes (the threshold to predict which class a bounding box is), therefore, to evaluate the accuracy of the model, ***it is necessary to calculate Precision and Recall at a specified IoU value*** (ex. survey the AP values at the threshold IoU=0.5)

Comparison of the models, based on AP@0.5 in the COCO dataset (COCO Val 2017)



Comparison of the models, based on runtime



Face detector

https://github.com/google-ai-edge/mediapipe-samples/tree/main/examples/face_detector

References

1) https://pytorch.org/tutorials/intermediate/torchvision_tutorial.html?highlight=maskrcnn_resnet50_fpn
2) https://pytorch.org/hub/nvidia_deeplearningexamples_ssd/
3) https://github.com/yhenon/pytorch-retinanet
4) https://github.com/xingyizhou/CenterNet
5) https://blog.roboflow.com/guide-to-yolo-models/
6) https://sefiks.com/2020/05/01/a-gentle-introduction-to-face-recognition-in-deep-learning/
7) https://ai.google.dev/edge/mediapipe/solutions/guide

Thank you!

Any question?